

FINAL PROJECT REPORT

OptiCrop – Smart Crop Recommendation System

Team Member	Role
Bala Sai Venkat Ramisetty	Team Lead
Eede Veda Srivalli	Member
Marneni Pranathi	Member
M Anunay	Member
Kadingu Uday Kumar	Member

Project Summary

A machine-learning-powered web application for crop recommendation using soil nutrients and environmental conditions.

1. INTRODUCTION

1.1 Project Overview

OptiCrop is a smart crop recommendation web application that combines a simple web interface with machine learning. Users enter soil macronutrient values (N, P and K) and environmental values (temperature, humidity, soil pH and rainfall). The application processes these seven features and recommends a crop. The result experience also presents crop details, ideal environmental conditions and growing guidance.

1.2 Purpose

The project aims to support better crop selection by providing fast, data-driven recommendations. It demonstrates how machine learning can be made accessible through a beginner-friendly web interface for agriculture-related decision support.

2. IDEATION PHASE

2.1 Problem Statement

Farmers and agricultural users may select crops without combining all relevant soil and climate factors. A mismatch between crop requirements and field conditions can reduce productivity and increase resource waste. The problem is to create an accessible system that accepts important measurable parameters and recommends a suitable crop using a trained classification model.

2.2 Empathy Map Canvas

Says: "Which crop is suitable for these conditions?" and "I need a simple recommendation."

Thinks: Crop choice affects cost, yield and risk.

Does: Checks soil values, observes weather and seeks local advice.

Feels: Uncertain when many environmental factors must be considered together.

Need: A simple, fast and understandable recommendation tool.

2.3 Brainstorming

Ideas considered include manual crop rules, a static crop information portal, weather-only recommendation and machine-learning prediction. The selected approach combines a Random Forest classifier with a web interface because it can handle multiple numerical features and produce an immediate crop class recommendation.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Awareness: user opens OptiCrop and understands the purpose. Input: user enters N, P, K, temperature, humidity, pH and rainfall. Processing: backend validates and sends the feature vector to the model. Result: system displays the recommended crop. Exploration: user reviews crop details, ideal conditions and growing tips. Decision support: user uses the recommendation together with local agricultural knowledge.

3.2 Solution Requirement

Functional requirements: accept seven inputs; validate data; submit prediction request; load trained model; predict one crop class; display recommendation; show crop information and guidance; provide workflow and model information.

Non-functional requirements: usability, responsive design, quick response time, reliability, maintainability and clear error handling.

3.3 Data Flow Diagram

Level 0: User → OptiCrop System → Crop Recommendation and Guidance.

Level 1: User Input → Validation/Preprocessing → Random Forest Model → Predicted Crop → Crop Information Mapping → Result Interface.

3.4 Technology Stack

Frontend: HTML, CSS and JavaScript. Backend: Python with Flask. Machine learning: scikit-learn Random Forest classifier. Data processing: pandas and NumPy. Model persistence: joblib or pickle. Development/testing: Python environment and modern web browser.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The system directly addresses the difficulty of combining multiple soil and climate factors. Instead of requiring users to interpret all seven variables manually, OptiCrop converts them into a recommendation and presents supporting information in the same interface.

4.2 Proposed Solution

Build a web application where users enter seven agricultural parameters. The backend preprocesses the input and sends it to a trained Random Forest model. The predicted crop is returned to the frontend and enriched with crop characteristics, ideal environmental conditions and growing tips.

4.3 Solution Architecture

Presentation Layer: HTML/CSS/JavaScript interface. Application Layer: Flask routes, validation and response handling. ML Layer: serialized Random Forest model. Data/Knowledge Layer: training dataset and crop descriptive information. Output Layer: recommendation result, crop details and guidance.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase 1 – Problem identification and requirement collection.
Phase 2 – Dataset understanding and preprocessing.
Phase 3 – Model training and evaluation.
Phase 4 – Flask backend integration.
Phase 5 – Frontend design and prediction form.
Phase 6 – Crop information and guidance sections.
Phase 7 – Functional testing, UI testing and documentation.
Phase 8 – Demo preparation and final presentation.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Functional tests verify required-field validation, numeric input handling, successful prediction submission, result rendering and navigation. Performance checks should measure page load time, model load behavior and prediction latency. The model should be loaded once at application startup rather than reloaded for every request. Boundary and invalid-input tests are important because unrealistic values can reduce recommendation usefulness.

7. RESULTS

7.1 Output Screenshots

The demo shows the complete user journey: input of crop parameters, prediction of rice, crop details, ideal conditions, growing tips, workflow explanation and model information.

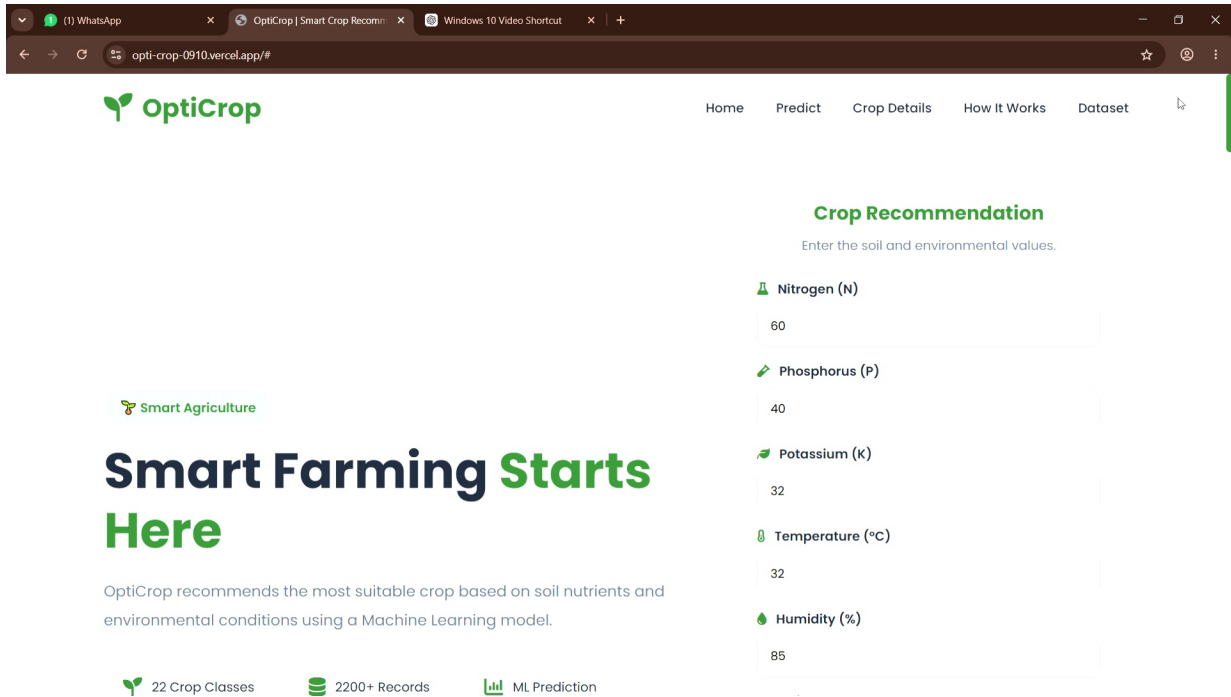


Figure 1: Home and input form

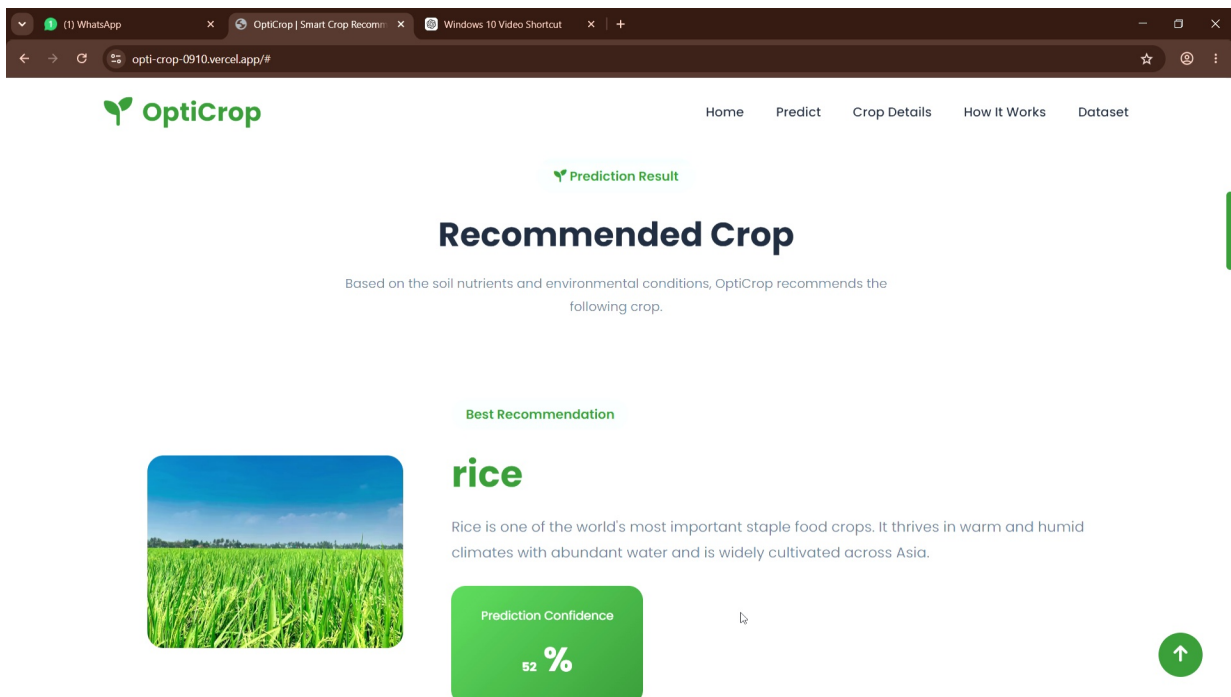


Figure 2: Recommended crop output

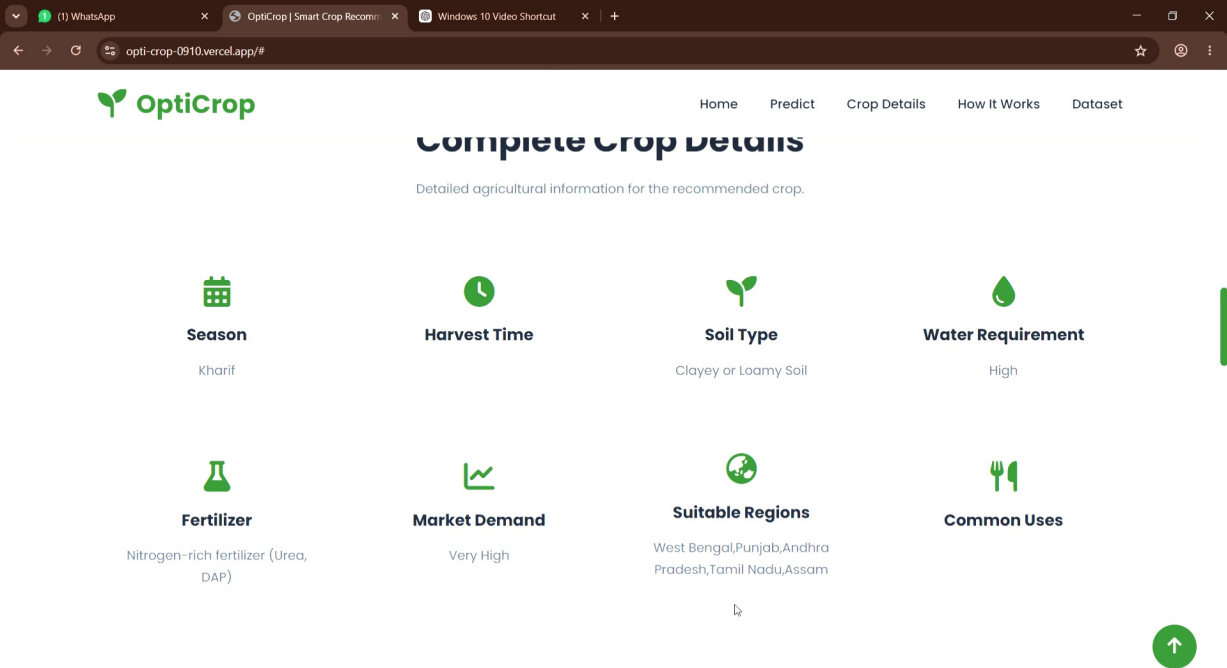


Figure 3: Crop details

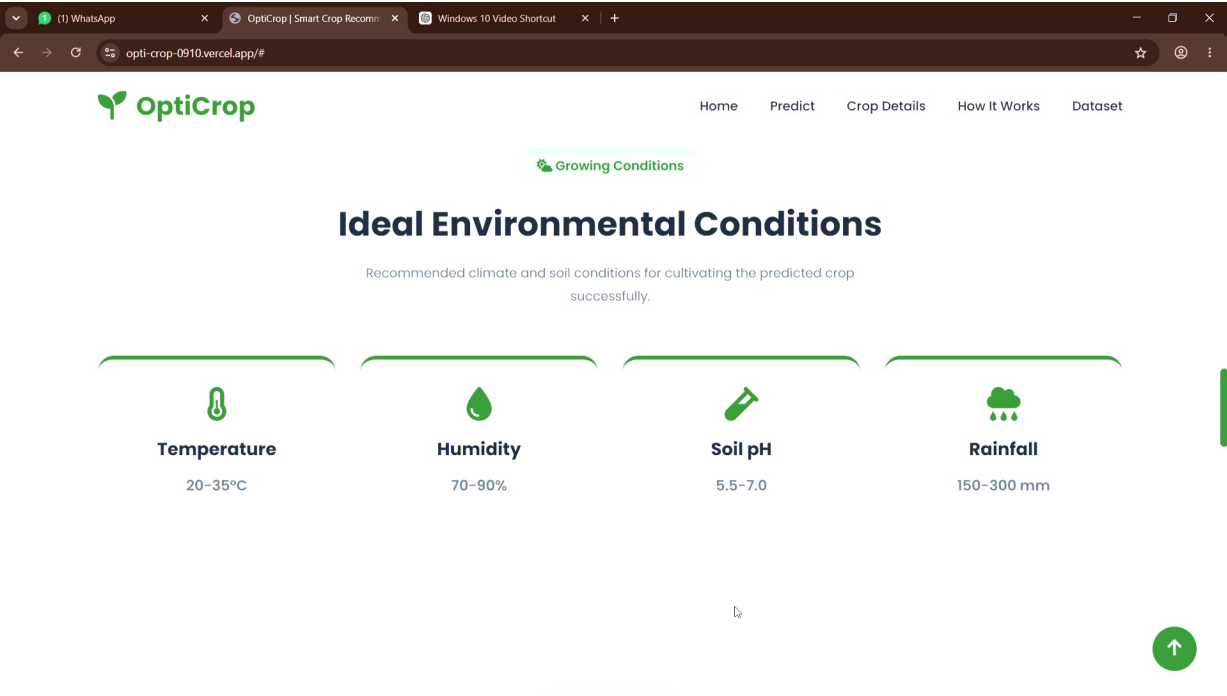


Figure 4: Ideal environmental conditions

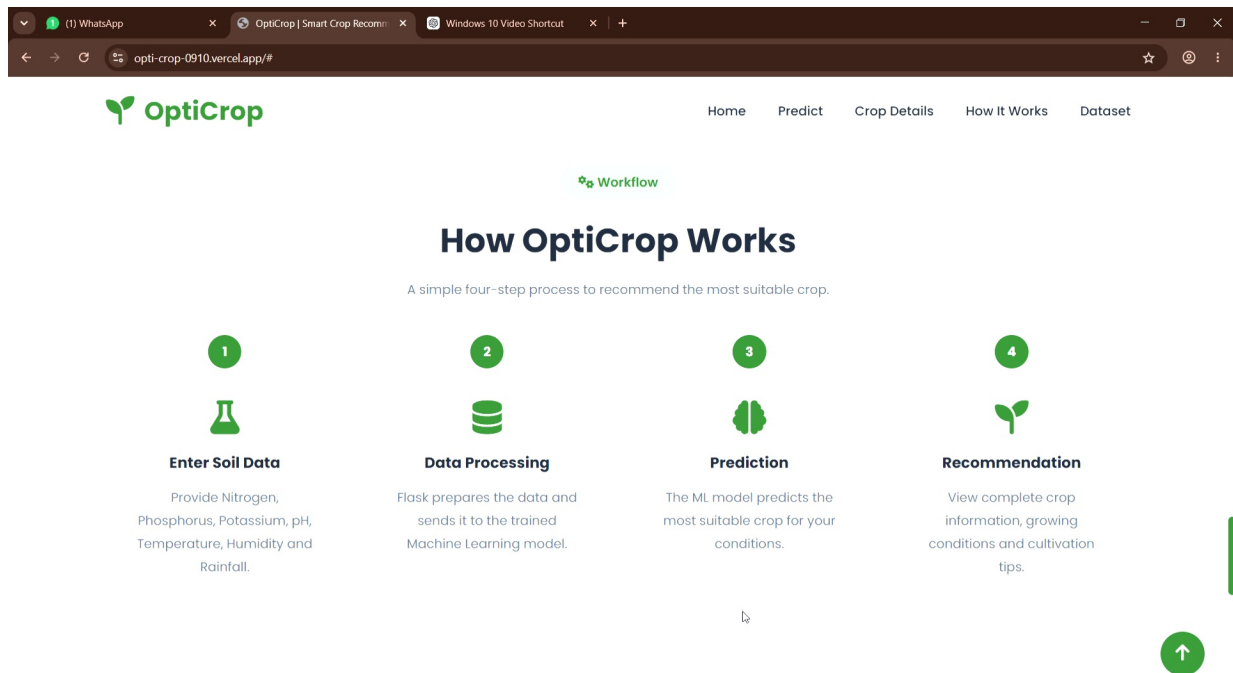


Figure 5: OptiCrop workflow

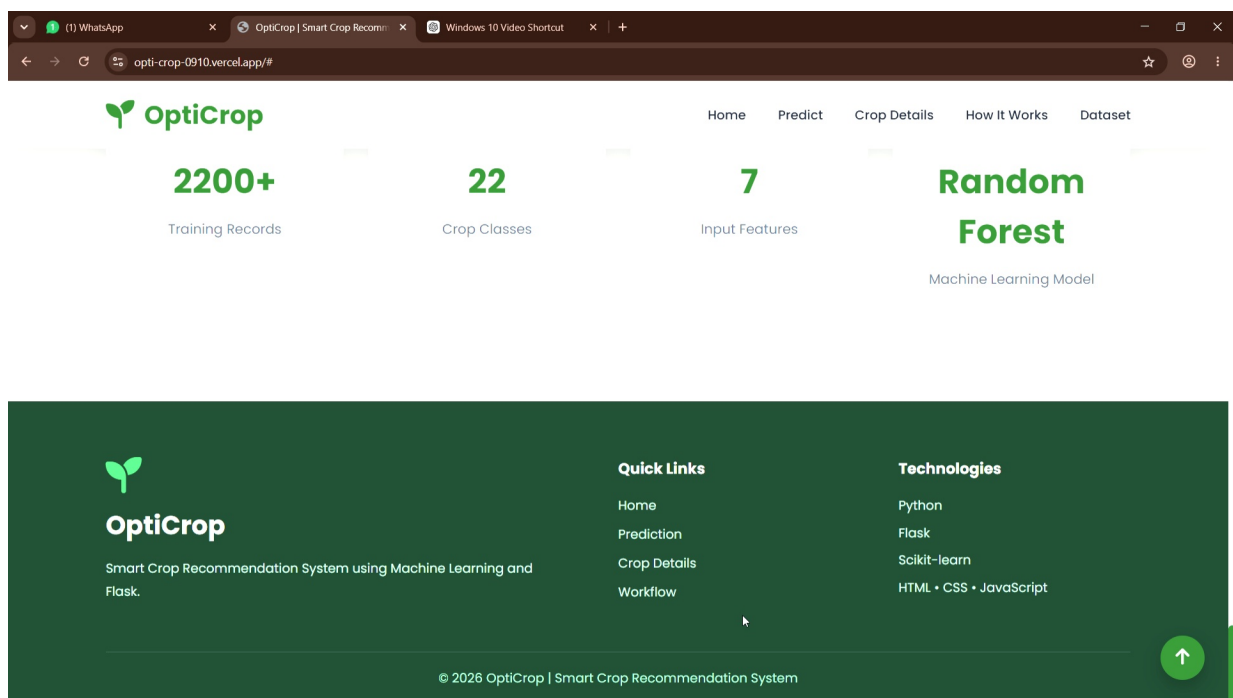


Figure 6: Model information

8. ADVANTAGES & DISADVANTAGES

Advantages

Fast recommendation; simple user interface; considers seven important features together; provides supporting crop information; scalable for additional crops and services; useful as an educational decision-support prototype.

Disadvantages

Predictions depend on training-data quality; does not account for every local factor such as pests, irrigation infrastructure, exact soil texture or market volatility; manually entered data may be inaccurate; the demo does

not show live weather or market integration. A wrong recommendation can have financial consequences, so real farming decisions should also use soil tests and qualified local agricultural advice.

9. CONCLUSION

Conclusion

OptiCrop demonstrates a practical integration of machine learning and web development for agriculture. By accepting seven soil and environmental parameters and using a Random Forest classifier, the system provides an immediate crop recommendation through a clear interface. The project also improves interpretability for users by presenting crop details, ideal conditions and growing tips after prediction.

10. FUTURE SCOPE

Future Scope

Future versions can add live weather data, GPS-based regional context, IoT soil sensors, multilingual support, fertilizer recommendations, pest and disease detection, yield prediction, market-price analytics, user accounts, prediction history, confidence scores and explainable-AI features.

11. APPENDIX

Source Code

Attach the final project source-code listing or repository reference here.

Dataset Link

Insert the exact dataset source link used by the team here. The website indicates 2200+ records, 22 crop classes and 7 input features.

GitHub & Project Demo Link

Insert the team's final GitHub repository and hosted project/demo links here.